Chapter 10

Distributed Database Management Systems

The Evolution of Distributed Database Management Systems

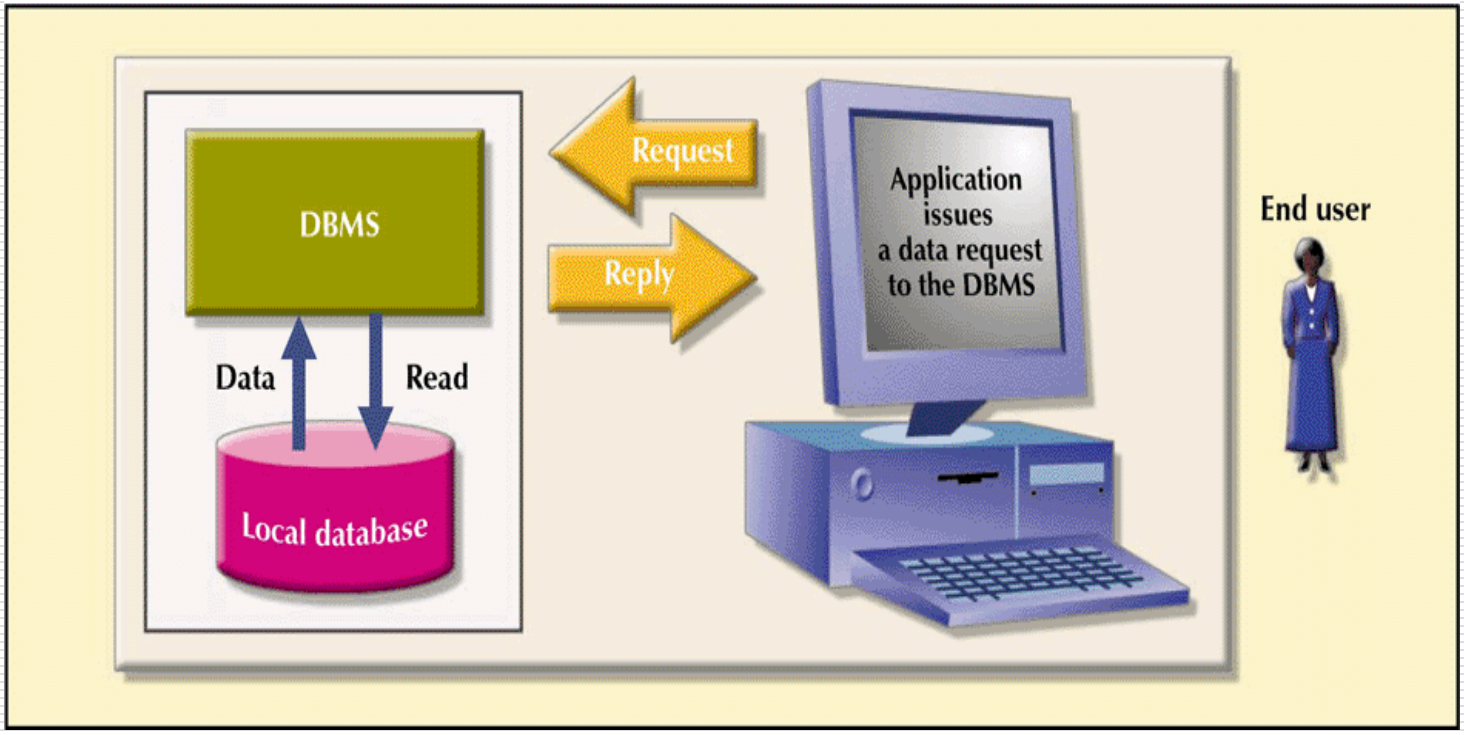
- Distributed database management system (DDBMS)
  - Governs storage and processing of logically related data over interconnected computer systems in which both data and processing functions are distributed among several sites

The Evolution of Distributed Database Management Systems (DDBMS)

- Centralized database required that corporate data be stored in a single central site
  - Performance degradation as number of remote sites grew
  - High cost to maintain large centralized DBs
  - Reliability problems with one, central site
- Dynamic business environment and centralized database's shortcomings spawned a demand for applications based on data access from different sources at multiple locations
  - Business operations became more decentralized geographically
  - Competition at global level
  - Rapid technological change in computers

Centralized Database Management System

FIGURE 10.1 CENTRALIZED DATABASE MANAGEMENT SYSTEM



DDBMS Advantages

- Data are located near "greatest demand" site
- Faster data access
- Faster data processing
- Growth facilitation
- Improved communications
- Reduced operating costs
- User-friendly interface
- Less danger of a single-point failure
- Processor independence

DDBMS Disadvantages

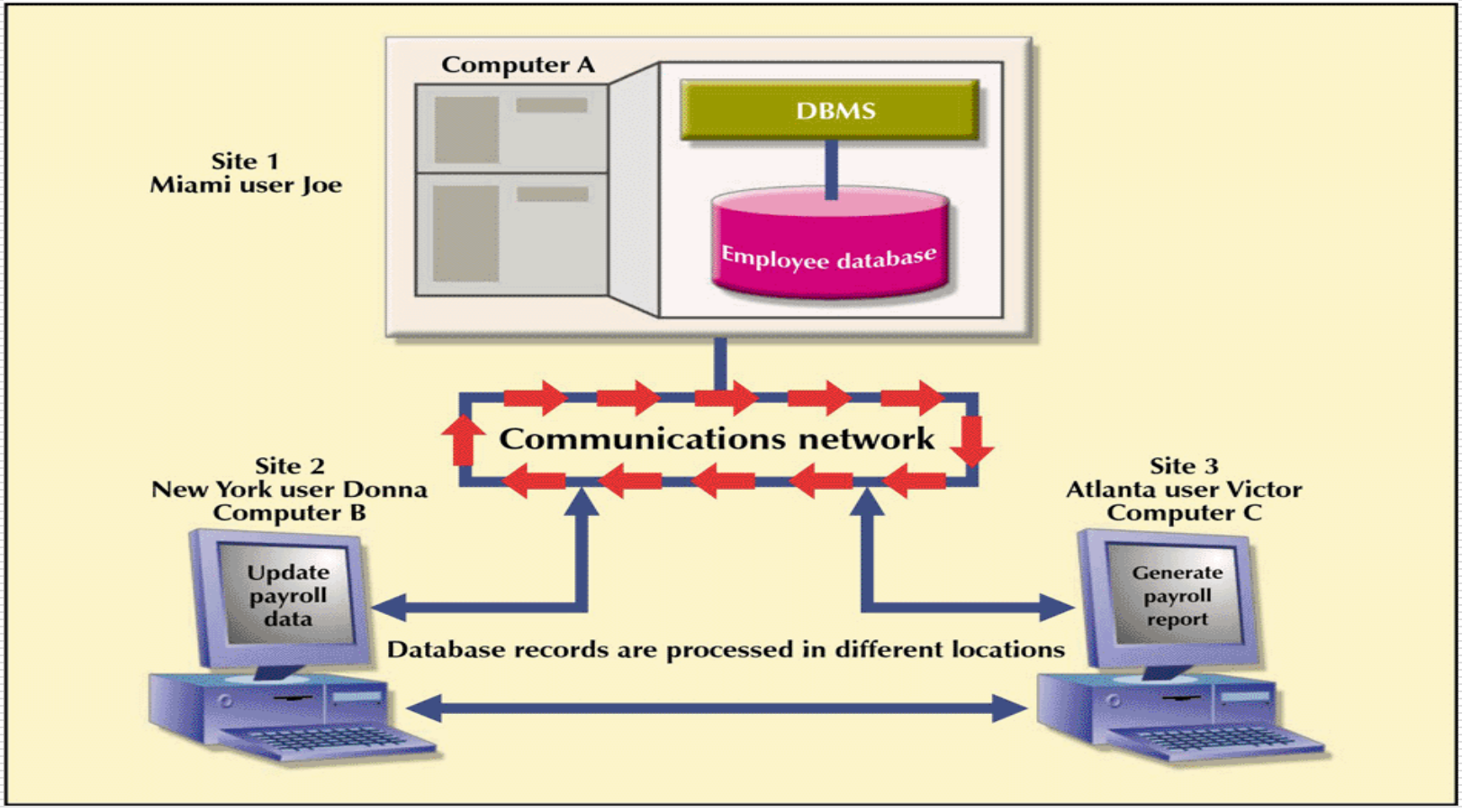
- Complexity of management and control
- Security
- Lack of standards
- Increased storage requirements
- Greater difficulty in managing the data environment
- Increased training cost

Distributed Processing vs Distributed Database

- Distributed processing – a database's logical processing is shared among two or more physically independent sites that are connected through a network
  - One computer performs I/O, data selection and validation while second computer creates reports
  - Uses a single-site database but the processing chores are shared among several sites
- Distributed database – stores a logically related database over two or more physically independent sites. The sites are connected via a network
  - Database is composed of database fragments which are located at different sites and may also be replicated among various sites

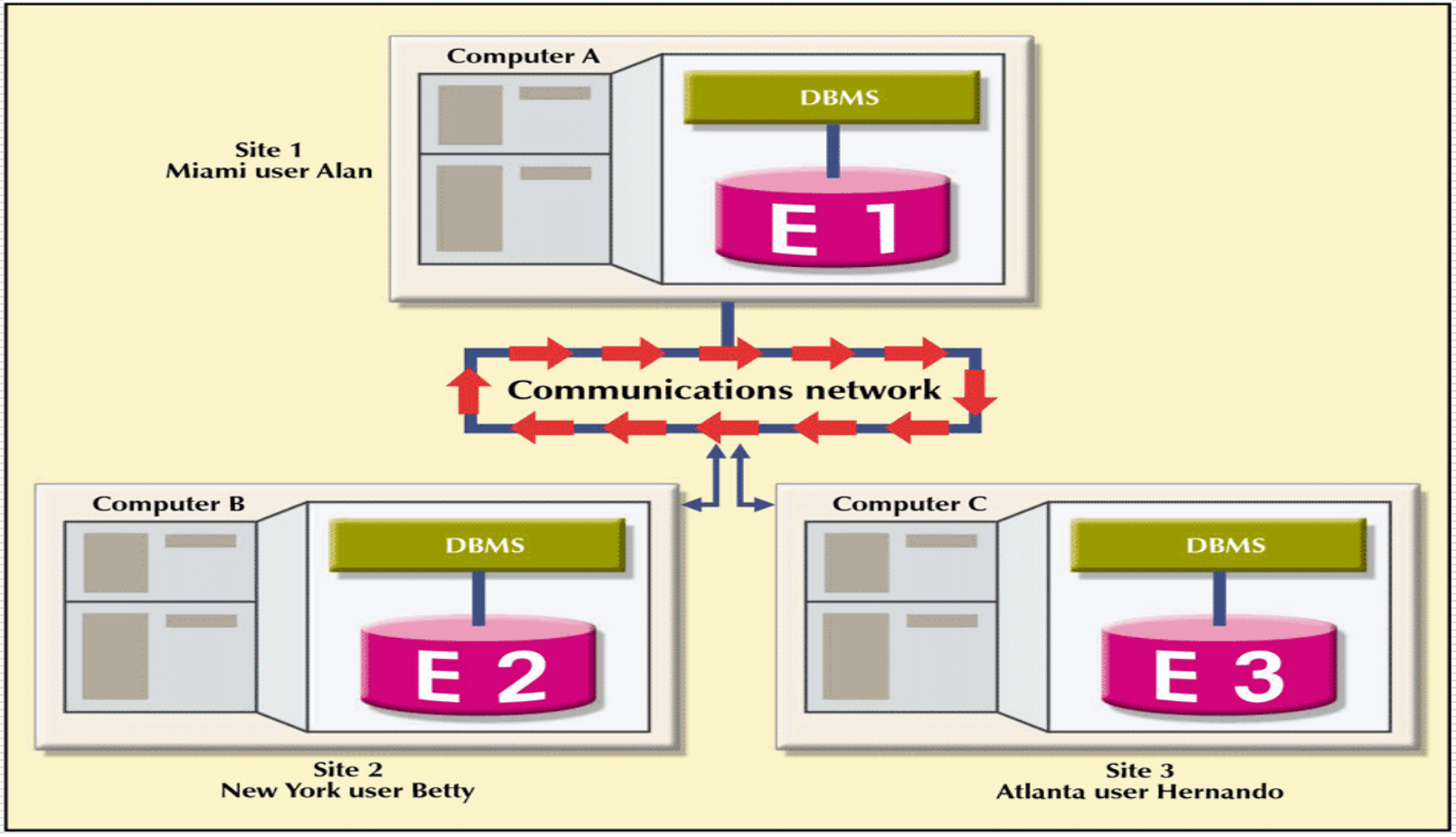
Distributed Processing Environment

FIGURE 10.2 DISTRIBUTED PROCESSING ENVIRONMENT



Distributed Database Environment

FIGURE 10.3 DISTRIBUTED DATABASE ENVIRONMENT



Characteristics of a DDBMS

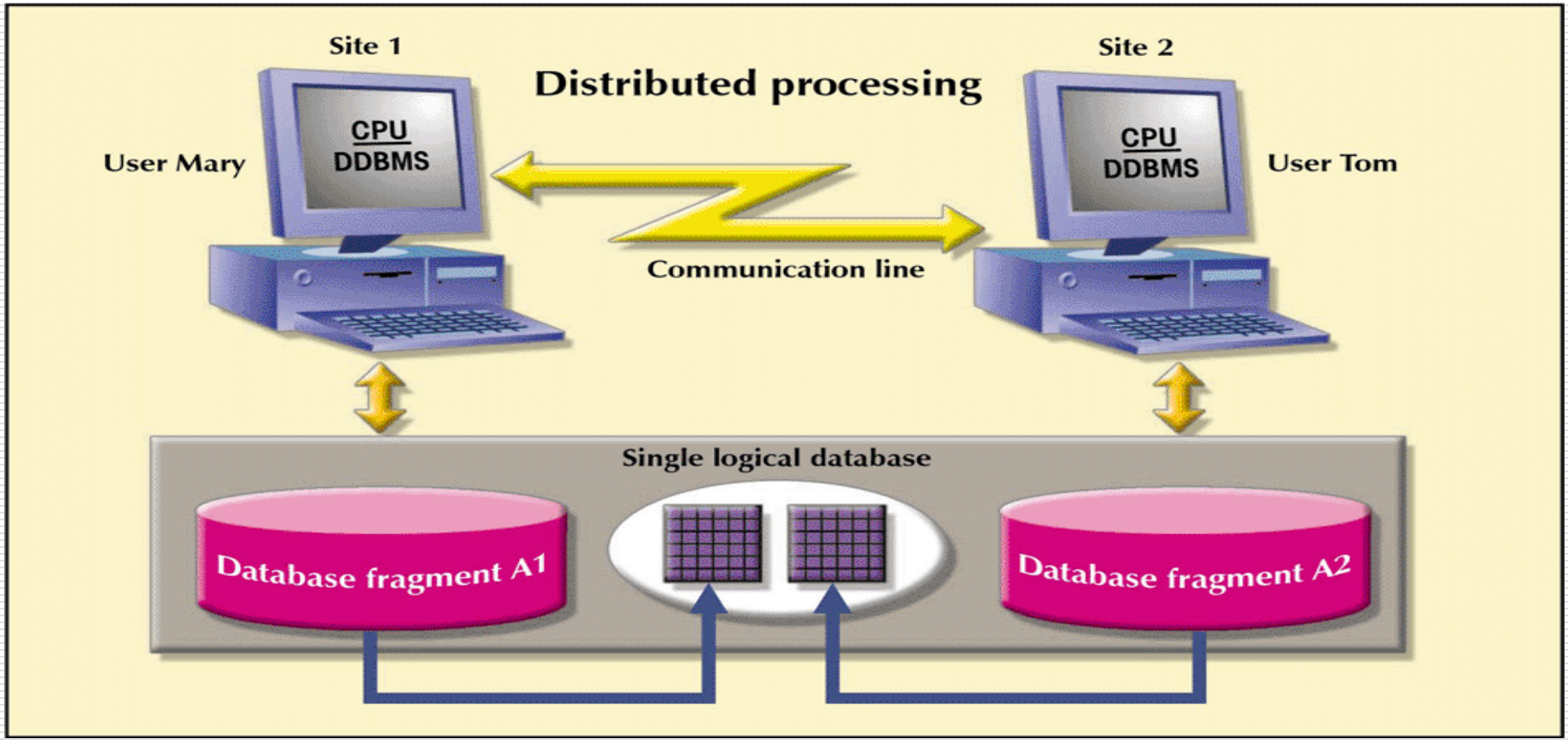
- Application interface
- Validation
- Transformation
- Query optimization
- Mapping
- I/O interface
- Formatting
- Security
- Backup and recovery
- DB administration
- Concurrency control
- Transaction management

Characteristics of Distributed Management Systems

- Must perform all the functions of a centralized DBMS
- Must handle all necessary functions imposed by the distribution of data and processing
- Must perform these additional functions *transparently* to the end user

A Fully Distributed Database Management System

FIGURE 10.4 A FULLY DISTRIBUTED DATABASE MANAGEMENT SYSTEM

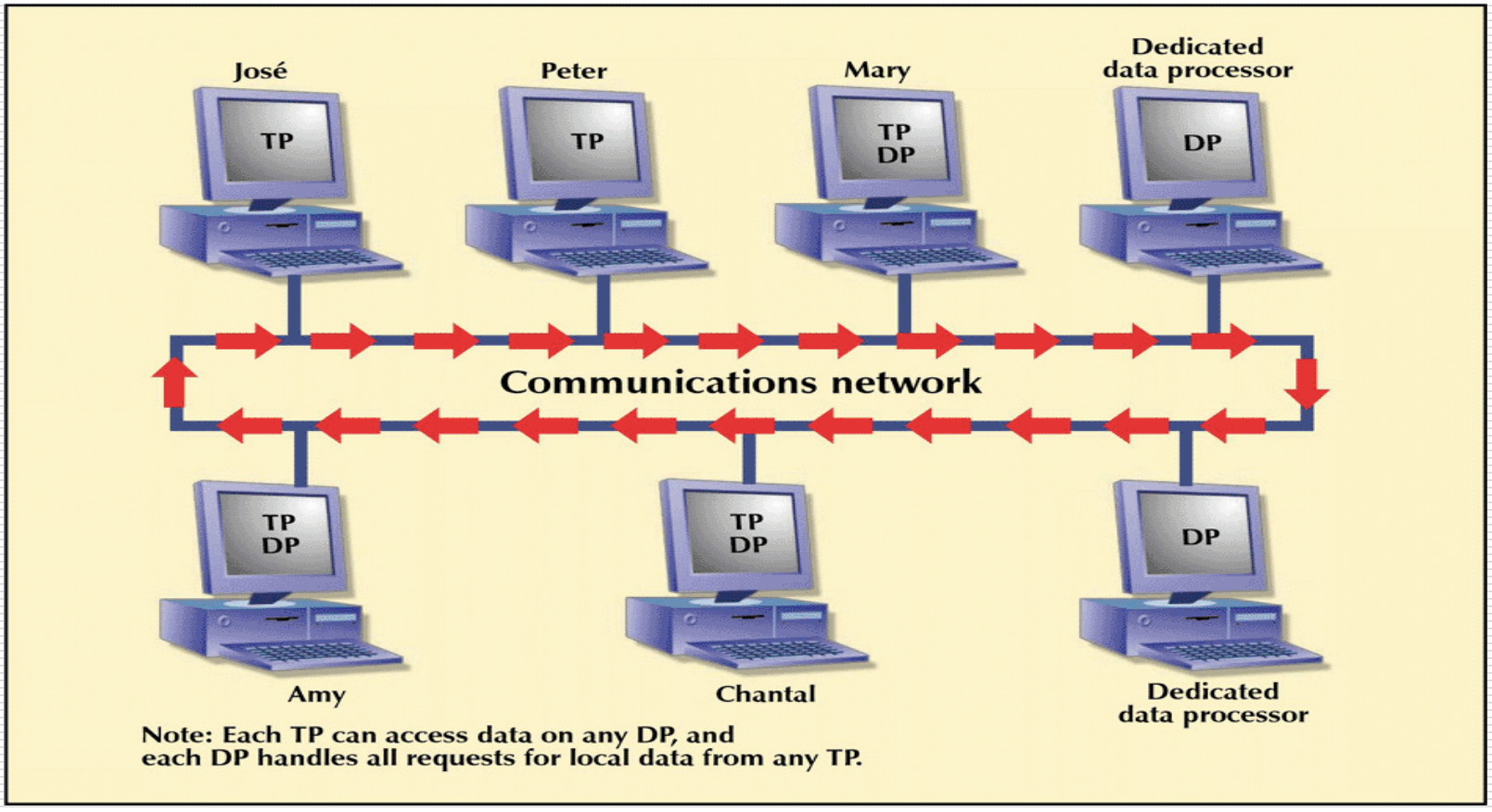


DDBMS Components

- Must include (at least) the following components:
  - Computer workstations
  - Network hardware and software
    - Allows all sites to interact and exchange data
  - Communications media
    - Carry the data from one workstation to another
  - Transaction processor (application processor or transaction manager)
    - Software component found in each computer that receives and processes the application's requests data
  - Data processor or data manager
    - Software component residing on each computer that stores and retrieves data located at the site
    - May even be a centralized DBMS
    - Communications between the TPs and DPs is made possible through a set of protocols used by the DDBMS

Distributed Database System Components

FIGURE 10.5 DISTRIBUTED DATABASE SYSTEM COMPONENTS



Database Systems: Levels of Data and Process Distribution

TABLE 10.1 DATABASE SYSTEMS: LEVELS OF DATA AND PROCESS DISTRIBUTION

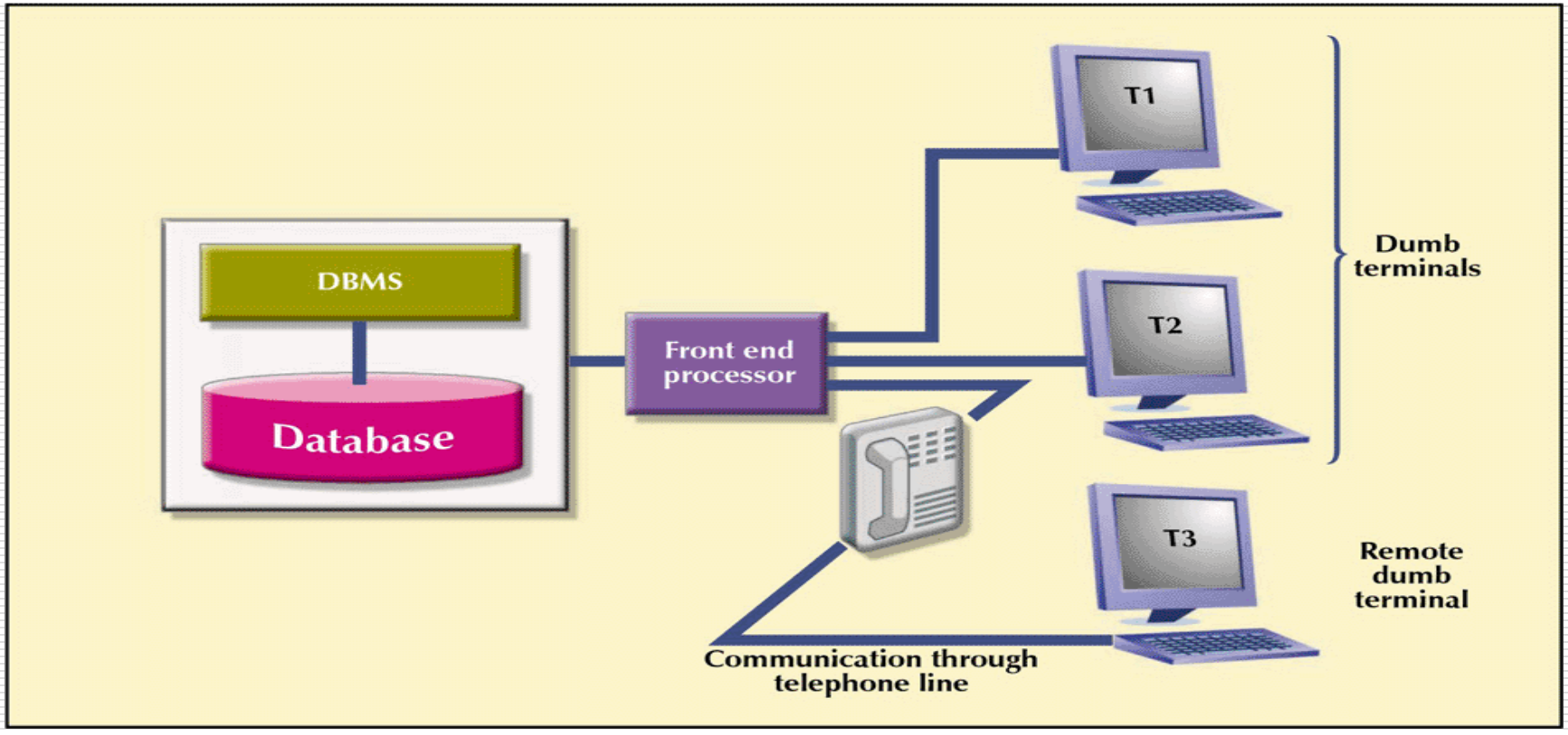
	SINGLE-SITE DATA	MULTIPLE-SITE DATA
Single-site process	Host DBMS (Mainframes)	Not applicable (Requires multiple processes)
Multiple-site process	File server Client/server DBMS (LAN DBMS)	Fully distributed Client/server DDBMS

Single-Site Processing, Single-Site Data (SPSD)

- All processing is done on single CPU or host computer (mainframe, midrange, or PC)
- All data are stored on host computer's local disk
- Processing cannot be done on end user's side of the system
- Typical of most mainframe and midrange computer DBMSs
- DBMS is located on the host computer, which is accessed by dumb terminals connected to it
- Also typical of the first generation of single-user microcomputer databases

Single-Site Processing, Single-Site Data (Centralized)

FIGURE 10.6 SINGLE-SITE PROCESSING, SINGLE-SITE DATA (CENTRALIZED)



Multiple-Site Processing, Single-Site Data (MPSD)

- Multiple processes run on different computers sharing a single data repository
- MPSD scenario requires a network file server running conventional applications that are accessed through a LAN
- Many multi-user accounting applications, running under a personal computer network, fit such a description

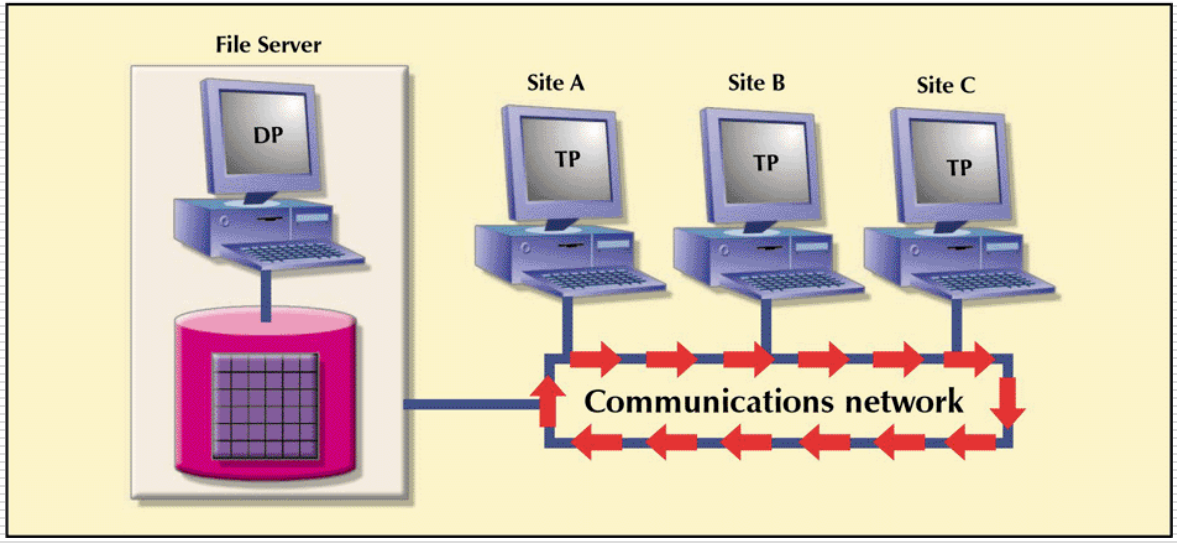
Multiple-Site Processing, Single-Site Data (MPSD)

- TP at each workstation acts only as a redirector to route all network data requests to the file server
- All record and file locking activity occurs at the end-user location
- All data selection, search and update functions takes place at the workstation. This requires entire files to travel through the network for processing at the workstation. This increases network traffic, slows response time and increases communication costs
  - To perform SELECT that results in 50 rows, a 10,000 row table must travel over the network to the end-user

Multiple-Site Processing, Single-Site Data (MPSD)

- In a variation of MPSD known as *client/server architecture*, all processing occurs at the server site, reducing the network traffic
- The processing is distributed; data can be located at multiple sites

FIGURE 10.7 MULTIPLE-SITE PROCESSING, SINGLE-SITE DATA

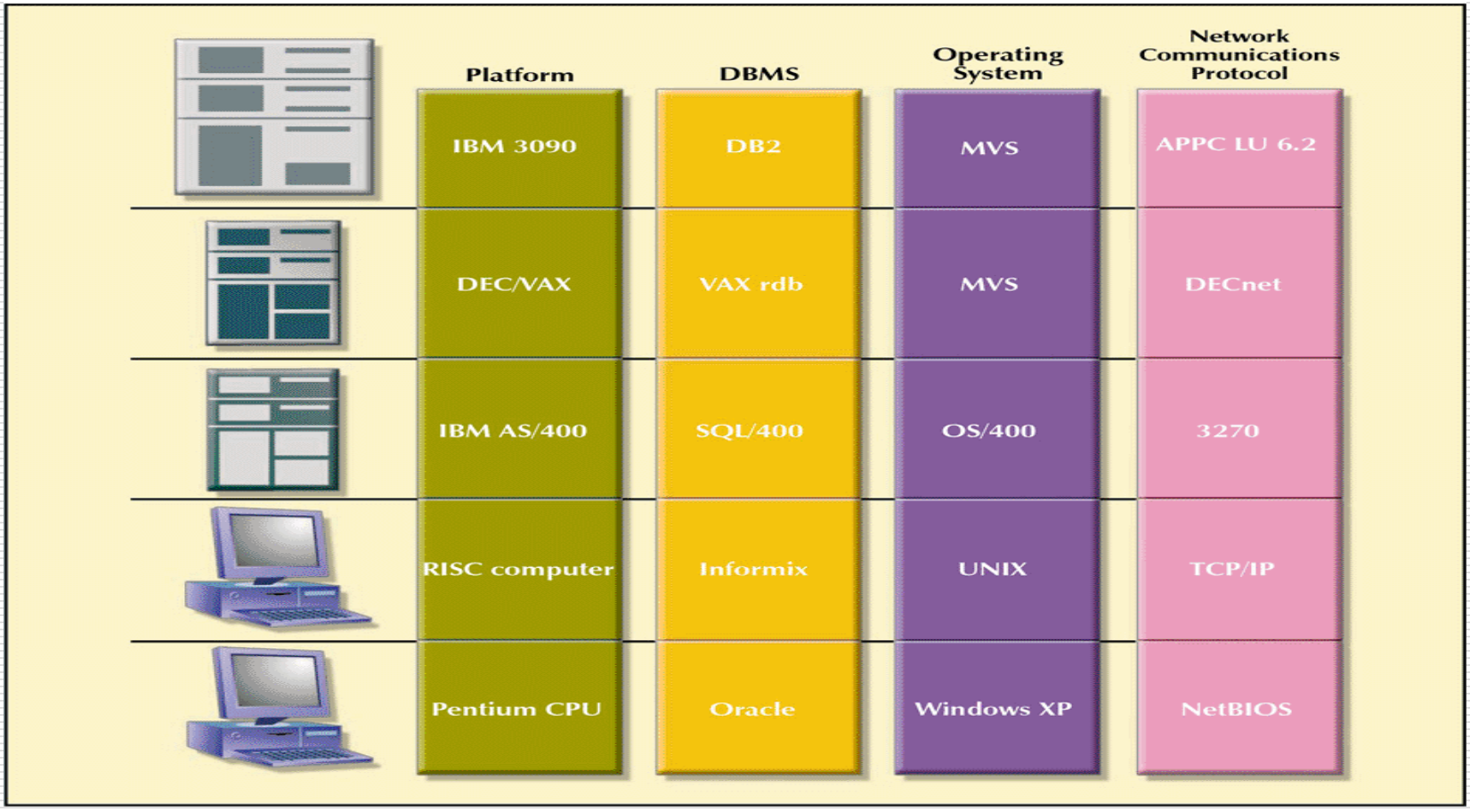


Multiple-Site Processing, Multiple-Site Data (MPMD)

- Fully distributed database management system with support for multiple data processors and transaction processors at multiple sites
- Classified as either homogeneous or heterogeneous
- Homogeneous DDBMSs
  - Integrate only one type of centralized DBMS over a network
  - The same DBMS will be running on different mainframes, minicomputers and microcomputers
- Heterogeneous DDBMSs
  - Integrate different types of centralized DBMSs over a network
- Fully heterogeneous DDBMS
  - Support different DBMSs that may even support different data models (relational, hierarchical, or network) running under different computer systems, such as mainframes and microcomputers
- No DDBMS currently provides full support for heterogeneous or fully heterogeneous DDBMSs

Heterogeneous Distributed Database Scenario

FIGURE 10.8 HETEROGENEOUS DISTRIBUTED DATABASE SCENARIO



Distributed Database Transparency Features

- Allow end user to feel like database's only user. User feels like they are working with a centralized database
- Features include:
  - *Distribution transparency* – user does not know where data is located and if replicated or partitioned
  - *Transaction transparency* – transaction can update at several network sites to ensure data integrity

Distributed Database Transparency Features

- *Failure transparency* – system continues to operate in the event of a node failure (other nodes pick up lost functionality)
- *Performance transparency* – allows system to perform as if it were a centralized DBMS. No performance degradation due to use of a network or platform differences
- *Heterogeneity transparency* – allows the integration of several different local DBMSs under a common schema

Distribution Transparency

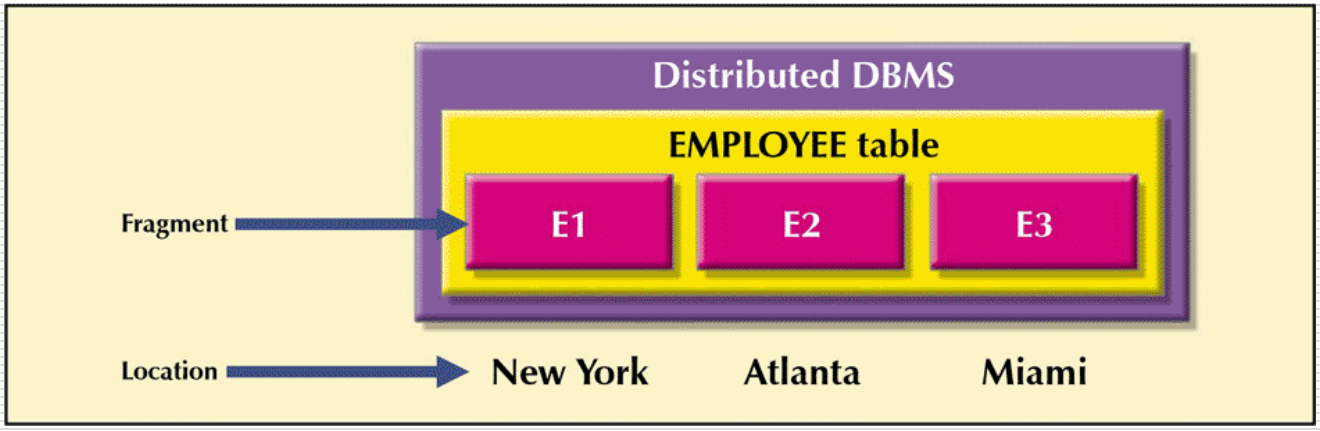
- Allows management of a physically dispersed database as though it were a centralized database
- Supported by a distributed data dictionary (DDD) which contains the description of the entire database as seen by the DBA
  - The DDD is itself distributed and replicated at the network nodes
- Three levels of distribution transparency are recognized:
  - *Fragmentation transparency* – user does not need to know if a database is partitioned; fragment names and/or fragment locations are not needed
  - *Location transparency* – fragment name, but not location, is required
  - *Local mapping transparency* – user must specify fragment name and location

A Summary of Transparency Features

TABLE 10.2 A SUMMARY OF TRANSPARENCY FEATURES

IF THE SQL STATEMENT REQUIRES:			
FRAGMENT NAME?	LOCATION NAME?	THEN THE DBMS SUPPORTS	LEVEL OF DISTRIBUTION TRANSPARENCY
Yes	Yes	Local mapping	Low
Yes	No	Location transparency	Medium
No	No	Fragmentation transparency	High

FIGURE 10.9 FRAGMENT LOCATIONS



Distribution Transparency

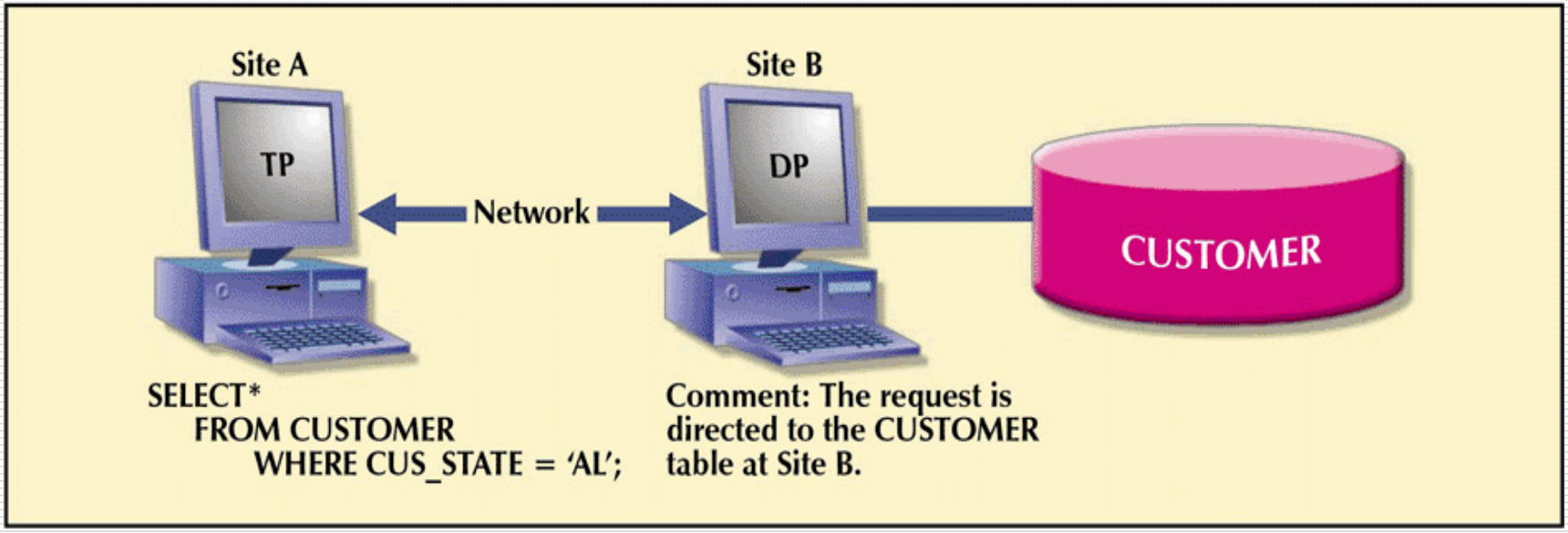
- The EMPLOYEE table is divided among three locations (no replication)
- Suppose an employee wants to find all employees with a birthdate prior to jan 1, 1940
  - Fragmentation transparency-
    - SELECT * FROM EMPLOYEE WHERE EMP_DOB < '01-JAN-1940';
  - Location transparency-
    - SELECT * FROM E1 WHERE EMP_DOB < '01-JAN-1940' UNION SELECT * FROM E2 … UNION SELECT * FROM E3…;
  - Local Mapping Transparency
    - SELECT * FROM E1 NODE NY WHERE EMP_DOB < '01-JAN-1940' UNION SELECT * FROM E2 NODE ATL … UNION SELECT * FROM E3 NODE MIA…;

Transaction Transparency

- Ensures database transactions will maintain distributed database's integrity and consistency
- A DDBMS transaction can update data stored in many different computers connected in a network
  - Transaction transparency ensures that the transaction will be completed only if all database sites involved in the transaction complete their part of the transaction

A Remote Request

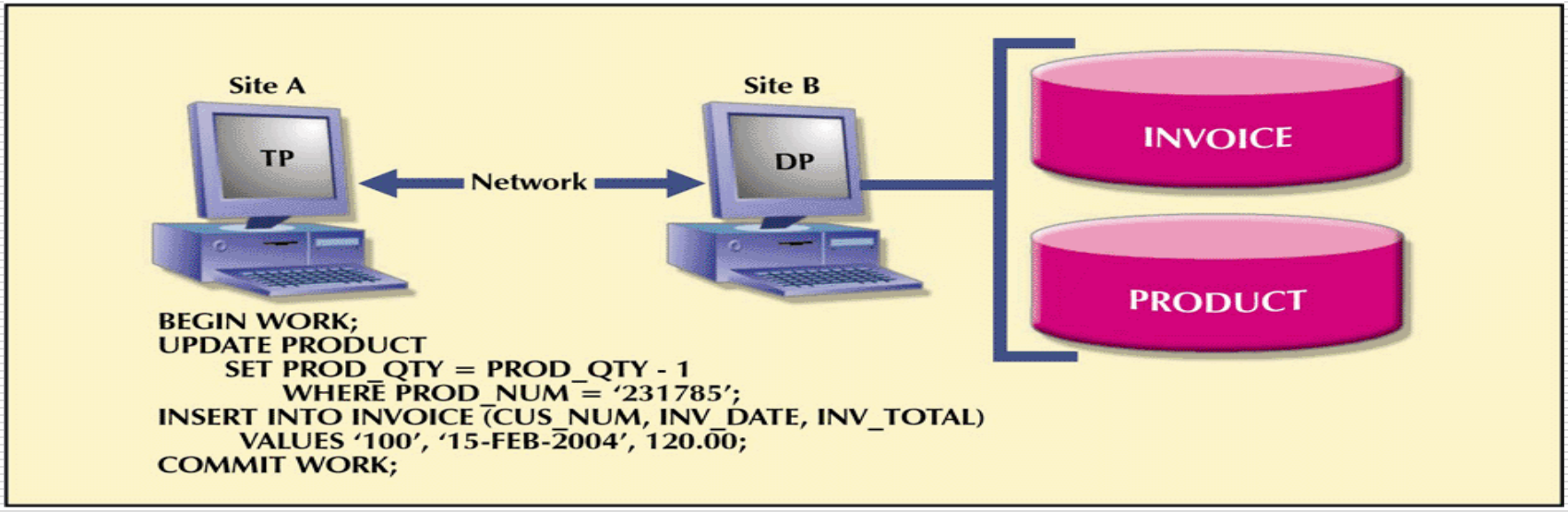
FIGURE 10.10 A REMOTE REQUEST



- Remote request
  - Lets a single SQL statement access data to be processed by a single remote database processor i.e., the SQL statement can reference data at only one remote site

A Remote Transaction

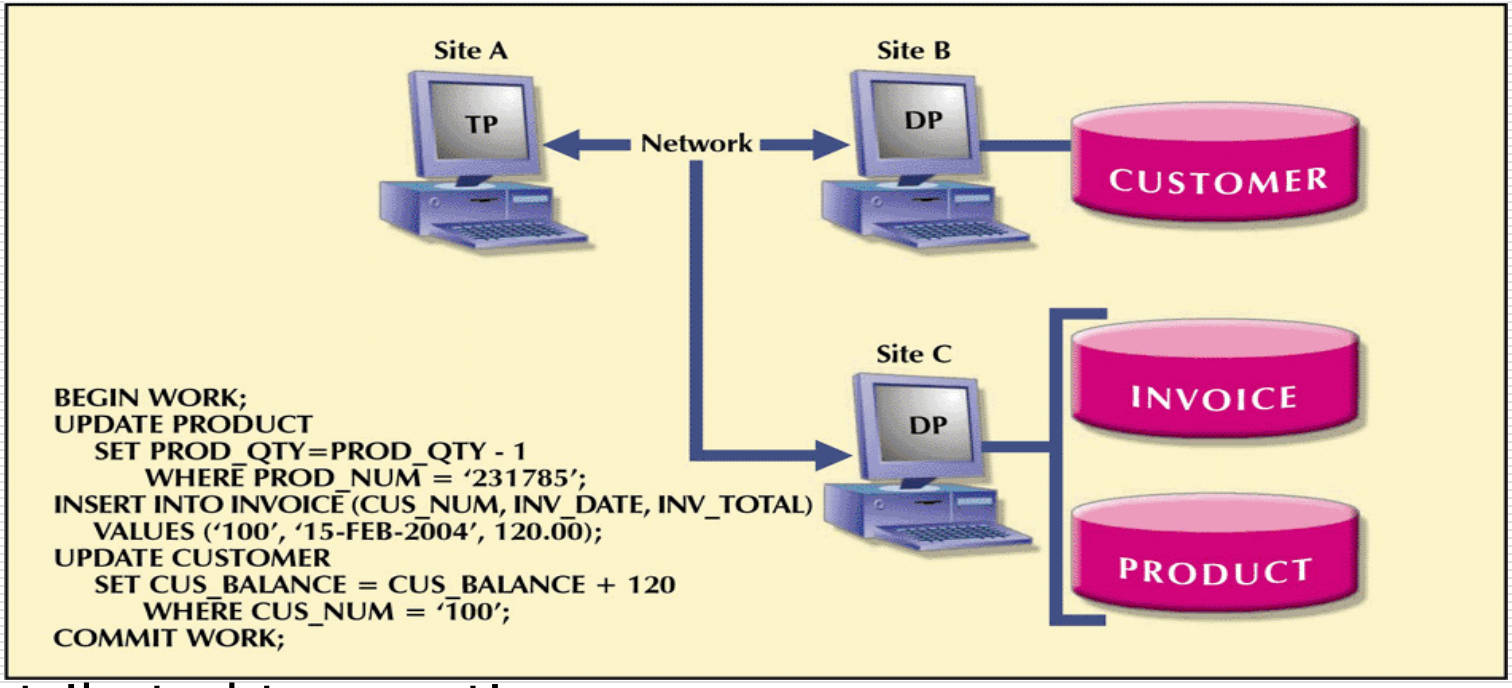
FIGURE 10.11 A REMOTE TRANSACTION



- Remote transaction
  - Accesses data at a single remote site
  - This transaction updates two tables
  - The remote transaction is sent to and executed at remote site B
  - The transaction can reference only one remote DP
  - Each SQL statement can reference only one remote DP at a time, and the entire transaction can reference and can be executed at only one remote DP

A Distributed Transaction

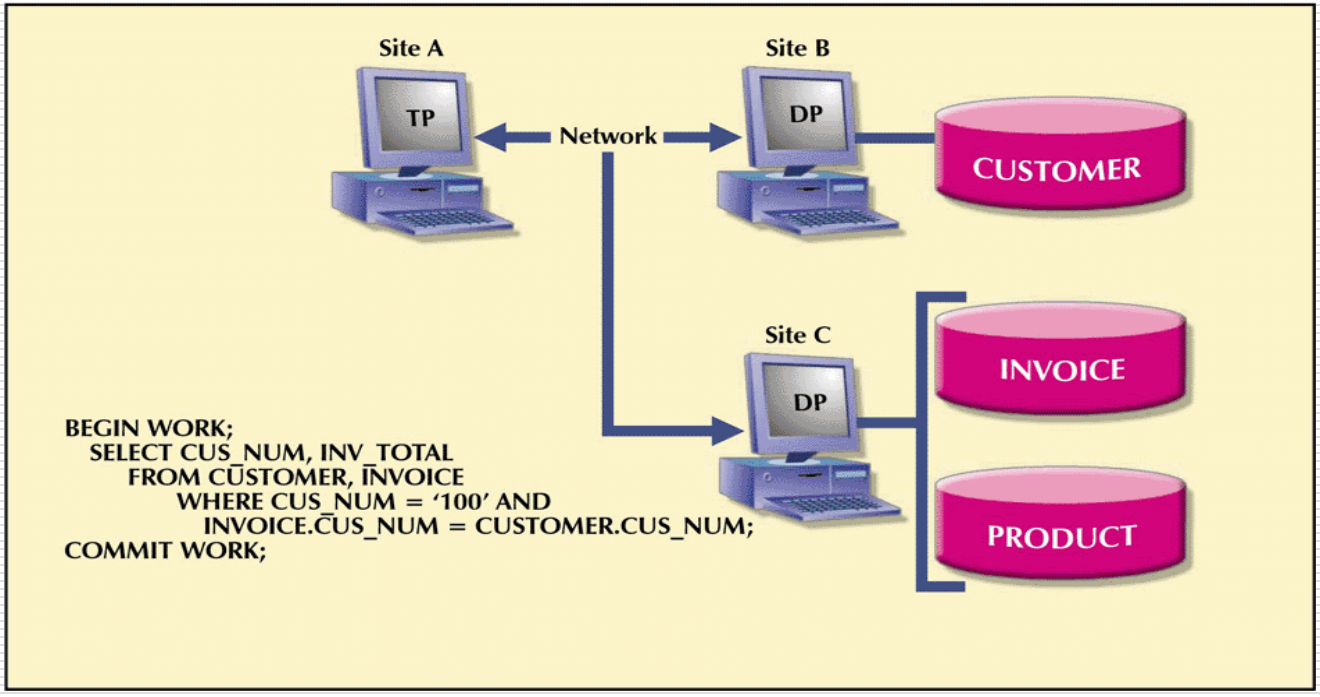
FIGURE 10.12 A DISTRIBUTED TRANSACTION



- Distributed transaction
  - Allows a transaction to reference several different (local or remote) DP sites
  - Each request can access only one remote site at a time
    - Does not support access to a table fragmented across multiple remote sites in one request

A Distributed Request

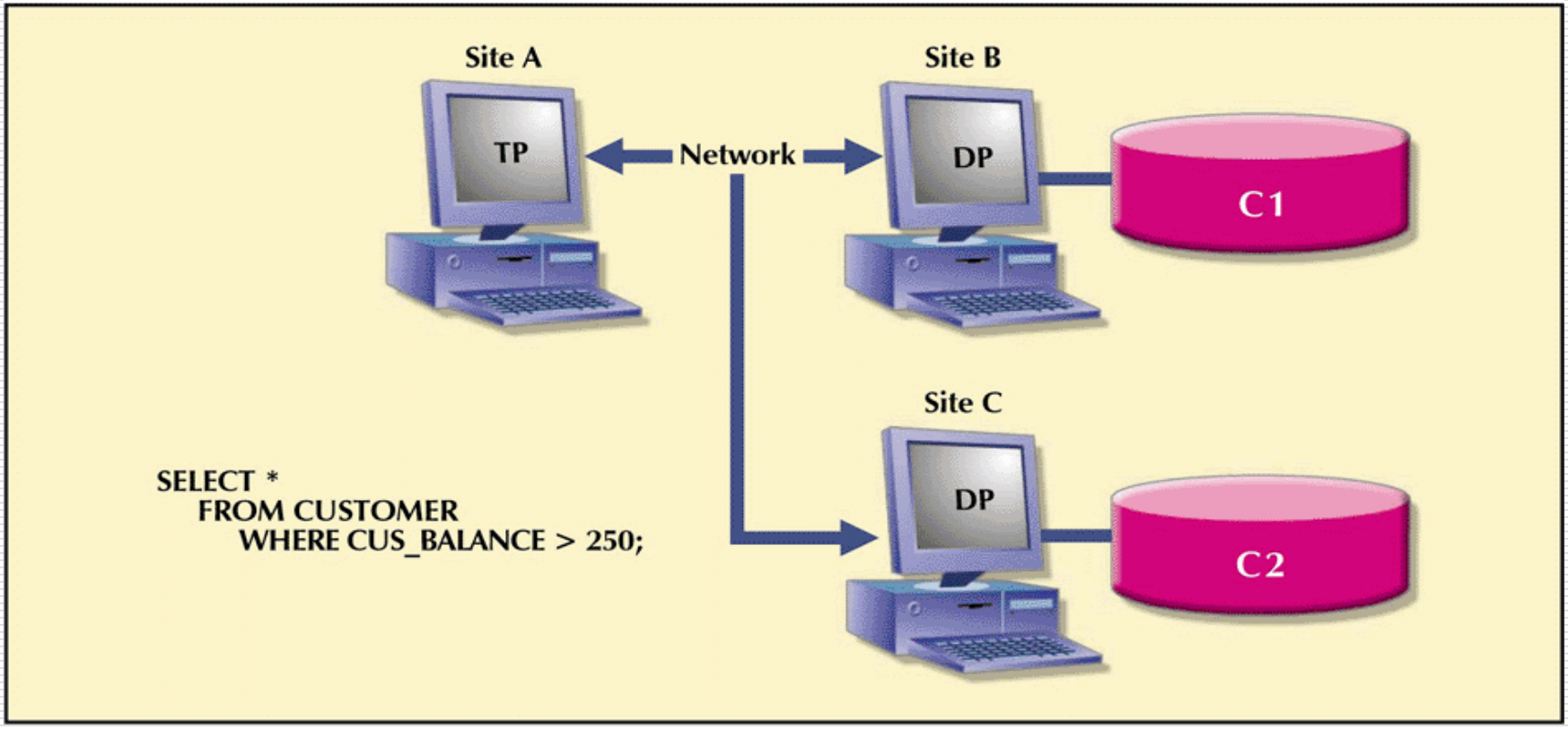
FIGURE 10.13 A DISTRIBUTED REQUEST



- Distributed request
  - Lets a single SQL statement reference data located at several different local or remote DP sites
    - The SELECT statement references two tables that are located at two different sites
  - Similarly, a table fragmented across two sites can be transparently queried in one SELECT (next slide)

Another Distributed Request

FIGURE 10.14 ANOTHER DISTRIBUTED REQUEST

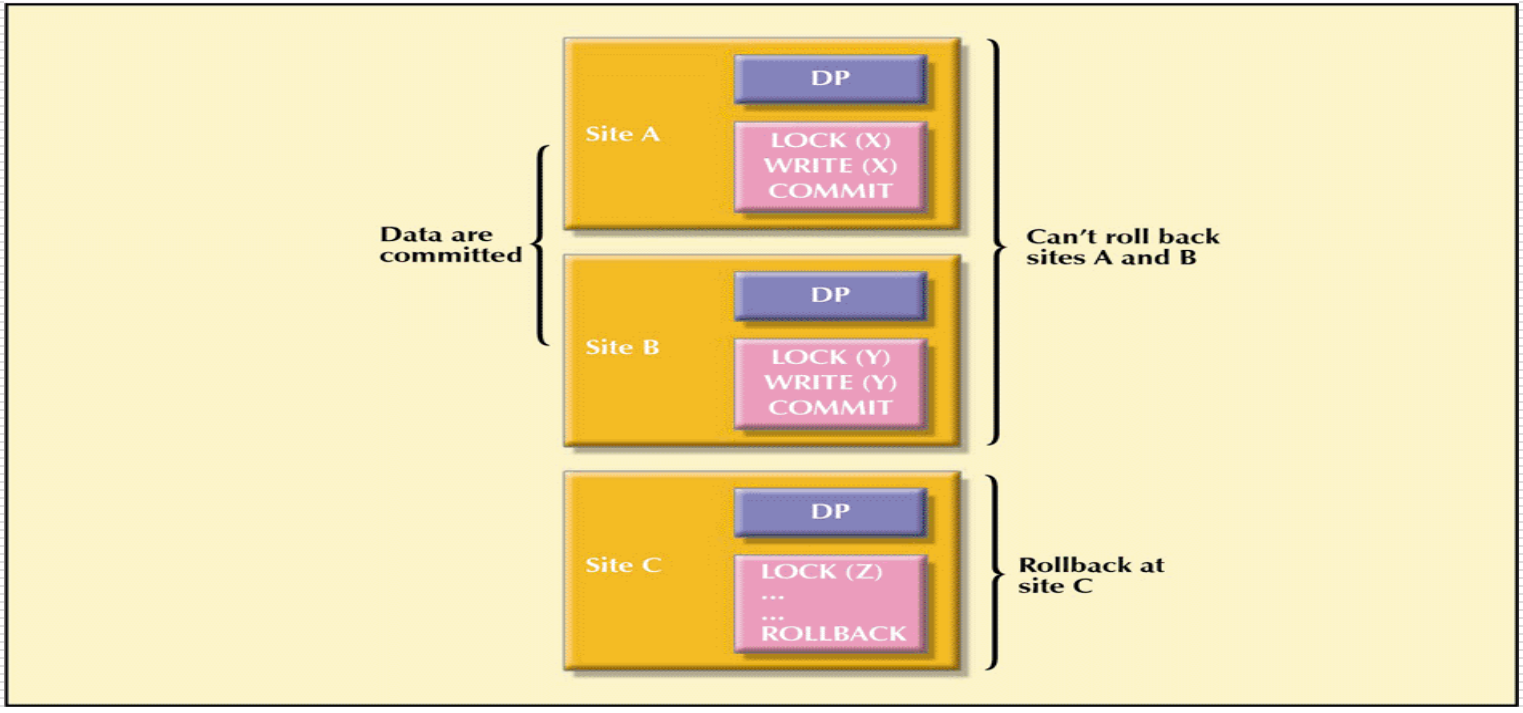


Distributed Concurrency Control

- Multisite, multiple-process operations are much more likely to create data inconsistencies and deadlocked transactions than are single-site systems
- The TP component of a DDBMS must ensure that all parts of the transaction, at all sites, are completed before a final COMMIT is issued to record the transaction

The Effect of a Premature COMMIT

FIGURE 10.15 THE EFFECT OF A PREMATURE COMMIT



- If one of the DPs did not commit and had to rollback while the other sites committed, the database would not be in a consistent state

Two-Phase Commit Protocol

- Distributed databases make it possible for a transaction to access data at several sites
- Final COMMIT must not be issued until all sites have committed their parts of the transaction
- Two-phase commit protocol requires each individual DP's transaction log entry be written before the database fragment is actually updated

Two-Phase Commit Protocol

- DO-UNDO-REDO protocol is used by the DP to roll back and/or roll forward transactions with the help of the system's transaction log entries
  - *DO* performs the operation and records the "before" and "after" values in the transaction log
  - *UNDO* reverses an operation, using the log entries written by the DO portion of the sequence
  - *REDO* redoes an operation, using the log entries written by the DO portion of the sequence
  - To ensure that the DO,UNDO and REDO operations can survive a system crash while they are being executed, a write-ahead protocol is used
    - This forces the log entry to be written to permanent storage before the actual operation takes place

Two-Phase Commit Protocol

- The two-phase commit protocol defines the operations between two types of nodes – the *coordinator* and one or more *subordinates*
- Phase I: Preparation
  - The coordinator sends a PREPARE TO COMMIT message to its subordinates
  - The subordinates receive the message, write the transaction log using the write-ahead protocol, and send an acknowledgement (YES/PREPARED TO COMMIT or NO/NOT PREPARED) message to the coordinator
  - The coordinator makes sure that all nodes are ready to commit or it aborts the action

Two-Phase Commit Protocol

- Phase II: The Final COMMIT
  - The coordinator broadcasts a COMMIT message to all subordinates and waits for replies
  - Each subordinate receives the COMMIT message, then updates the database using the DO protocol
  - The subordinates reply with a COMMITTED or NOT COMMITTED message to the coordinator
  - If one or more subordinates did not commit, the coordinator sends an ABORT message, forcing them to UNDO all changes
    - The information necessary to recover the database is in the transaction log and the database can be recovered with the DO-UNDO-REDO protocol

Distributed Database Design

- Data fragmentation:
  - How to partition the database into fragments
- Data replication:
  - Which fragments to replicate
- Data allocation:
  - Where to locate those fragments and replicas

Data Fragmentation

- Breaks single object into two or more segments or fragments
- Each fragment can be stored at any site over a computer network
- Information about data fragmentation is stored in the distributed data catalog (DDC), from which it is accessed by the TP to process user requests

Data Fragmentation Strategies

- Horizontal fragmentation:
  - Division of a relation into subsets (fragments) of tuples (rows)
- Vertical fragmentation:
  - Division of a relation into attribute (column) subsets
- Mixed fragmentation:
  - Combination of horizontal and vertical strategies

A Sample CUSTOMER Table

FIGURE 10.16 A SAMPLE CUSTOMER TABLE

Table name: CUSTOMER

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
10	Sinex, Inc.	12 Main St.	TN	$3,500.00	$2,700.00	3	$1,245.00
11	Martin Corp.	321 Sunset Blvd.	FL	$6,000.00	$1,200.00	1	$0.00
12	Mynux Corp.	910 Eagle St.	TN	$4,000.00	$3,500.00	3	$3,400.00
13	BTBC, Inc.	Rue du Monde	FL	$6,000.00	$5,890.00	3	$1,090.00
14	Victory, Inc.	123 Maple St.	FL	$1,200.00	$550.00	1	$0.00
15	NBCC Corp.	909 High Ave.	GA	$2,000.00	$350.00	2	$50.00

Horizontal Fragmentation of the CUSTOMER Table by State

TABLE 10.3 HORIZONTAL FRAGMENTATION OF THE CUSTOMER TABLE BY STATE

FRAGMENT NAME	LOCATION	CONDITION	NODE NAME	CUSTOMER NUMBERS	NUMBER OF ROWS
CUST_H1	Tennessee	CUS_STATE = 'TN'	NAS	10, 12	2
CUST_H2	Georgia	CUS_STATE = 'GA'	ATL	15	1
CUST_H3	Florida	CUS_STATE = 'FL'	TAM	11, 13, 14	3

FIGURE 10.17 TABLE FRAGMENTS IN THREE LOCATIONS

Table name: CUST_H1 **Location: Tennessee** **Node: NAS**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
10	Sinex, Inc.	12 Main St.	TN	$3,500.00	$2,700.00	3	$1,245.00
12	Mynux Corp.	910 Eagle St.	TN	$4,000.00	$3,500.00	3	$3,400.00

Table name: CUST_H2 **Location: Georgia** **Node: ALT**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
15	NBCC Corp.	909 High Ave.	GA	$2,000.00	$350.00	2	$50.00

Table name: CUST_H3 **Location: Florida** **Node: TAM**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
11	Martin Corp.	321 Sunset Blvd.	FL	$6,000.00	$1,200.00	1	$0.00
13	BTBC, Inc.	Rue du Monde	FL	$6,000.00	$5,890.00	3	$1,090.00
14	Victory, Inc.	123 Maple St.	FL	$1,200.00	$550.00	1	$0.00

Vertically Fragmented Table Contents

FIGURE 10.18 VERTICALLY FRAGMENTED TABLE CONTENTS

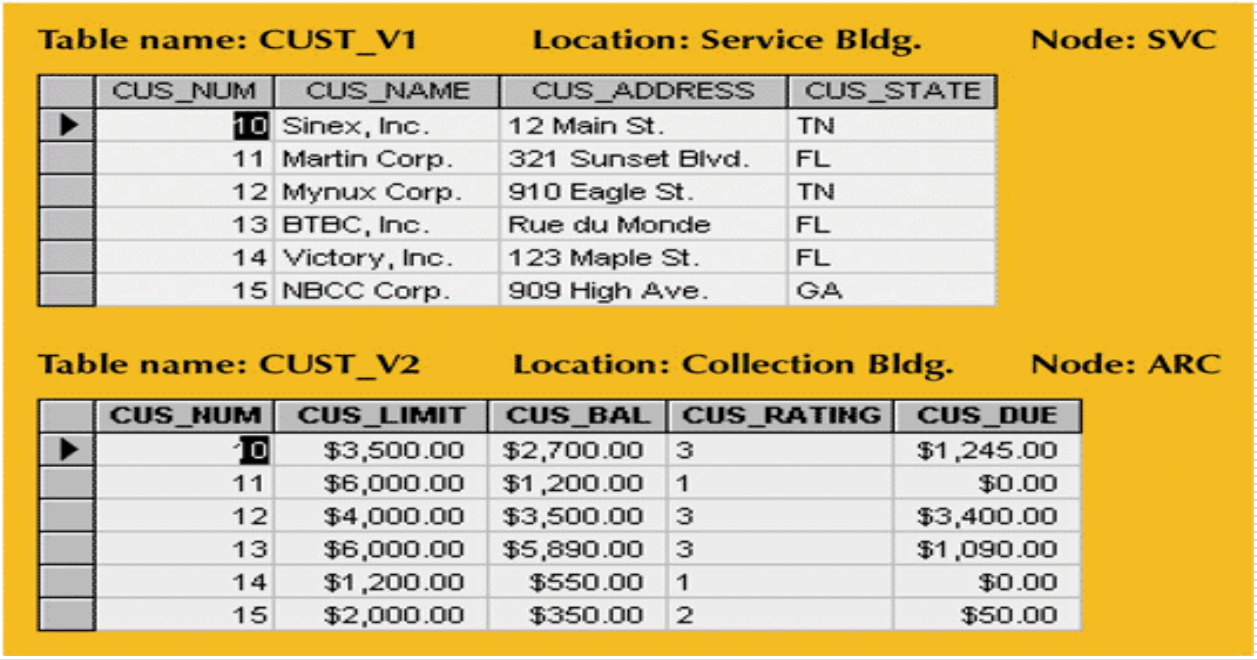


Table name: CUST_V1 Location: Service Bldg. Node: SVC

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE
10	Sinex, Inc.	12 Main St.	TN
11	Martin Corp.	321 Sunset Blvd.	FL
12	Mynux Corp.	910 Eagle St.	TN
13	BTBC, Inc.	Rue du Monde	FL
14	Victory, Inc.	123 Maple St.	FL
15	NBCC Corp.	909 High Ave.	GA

Table name: CUST_V2 Location: Collection Bldg. Node: ARC

CUS_NUM	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
10	$3,500.00	$2,700.00	3	$1,245.00
11	$6,000.00	$1,200.00	1	$0.00
12	$4,000.00	$3,500.00	3	$3,400.00
13	$6,000.00	$5,890.00	3	$1,090.00
14	$1,200.00	$550.00	1	$0.00
15	$2,000.00	$350.00	2	$50.00

Two separate areas in the company use different fields of the table in the daily activities – the SERVICE dept and the COLLECTIONS dept

Mixed Fragmentation of the CUSTOMER Table

TABLE 10.5 MIXED FRAGMENTATION OF THE CUSTOMER TABLE

FRAGMENT NAME	LOCATION	HORIZONTAL CRITERIA	NODE NAME	RESULTING ROWS AT SITE	VERTICAL CRITERIA ATTRIBUTES AT EACH FRAGMENT
CUST_M1	TN-Service	CUS_STATE = 'TN'	NAS-S	10, 12	CUS_NUM, CUS_NAME CUS_ADDRESS, CUS_STATE
CUST_M2	TN-Collection	CUS_STATE = 'TN'	NAS-C	10, 12	CUS_NUM, CUS_LIMIT, CUS_BAL, CUS_RATING, CUS_DUE
CUST_M3	GA-Service	CUS_STATE = 'GA'	ATL-S	15	CUS_NUM, CUS_NAME CUS_ADDRESS, CUS_STATE
CUST_M4	GA-Collection	CUS_STATE = 'GA'	ATL-C	15	CUS_NUM, CUS_LIMIT, CUS_BAL, CUS_RATING, CUS_DUE
CUST_M5	FL-Service	CUS_STATE = 'FL'	TAM-S	11, 13, 14	CUS_NUM, CUS_NAME CUS_ADDRESS, CUS_STATE
CUST_M6	FL-Collection	CUS_STATE = 'FL'	TAM-C	11, 13, 14	CUS_NUM, CUS_LIMIT, CUS_BAL, CUS_RATING, CUS_DUE

The table is divided horizontally by the three states and within each state there is a vertical fragmentation by department

Table Contents After the Mixed Fragmentation Process

FIGURE 10.19 TABLE CONTENTS AFTER THE MIXED FRAGMENTATION PROCESS

Table name: CUST_M1 **Location: TN-Service** **Node: NAS-S**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE
10	Sinex, Inc.	12 Main St.	TN
12	Mynux Corp.	910 Eagle St.	TN

Table name: CUST_M2 **Location: TN-Collection** **Node: NAS-C**

CUS_NUM	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
10	$3,500.00	$2,700.00	3	$1,245.00
12	$4,000.00	$3,500.00	3	$3,400.00

Table name: CUST_M3 **Location: GA-Service** **Node: ATL-S**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE
15	NBCC Corp.	909 High Ave.	GA

Table name: CUST_M4 **Location: GA-Collection** **Node: ATL-C**

CUS_NUM	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
15	$2,000.00	$350.00	2	$50.00

Table name: CUST_M5 **Location: FL-Service** **Node: TAM-S**

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_STATE
11	Martin Corp.	321 Sunset Blvd.	FL
13	BTBC, Inc.	Rue du Monde	FL
14	Victory, Inc.	123 Maple St.	FL

Table name: CUST_M6 **Location: FL-Collection** **Node: TAM-C**

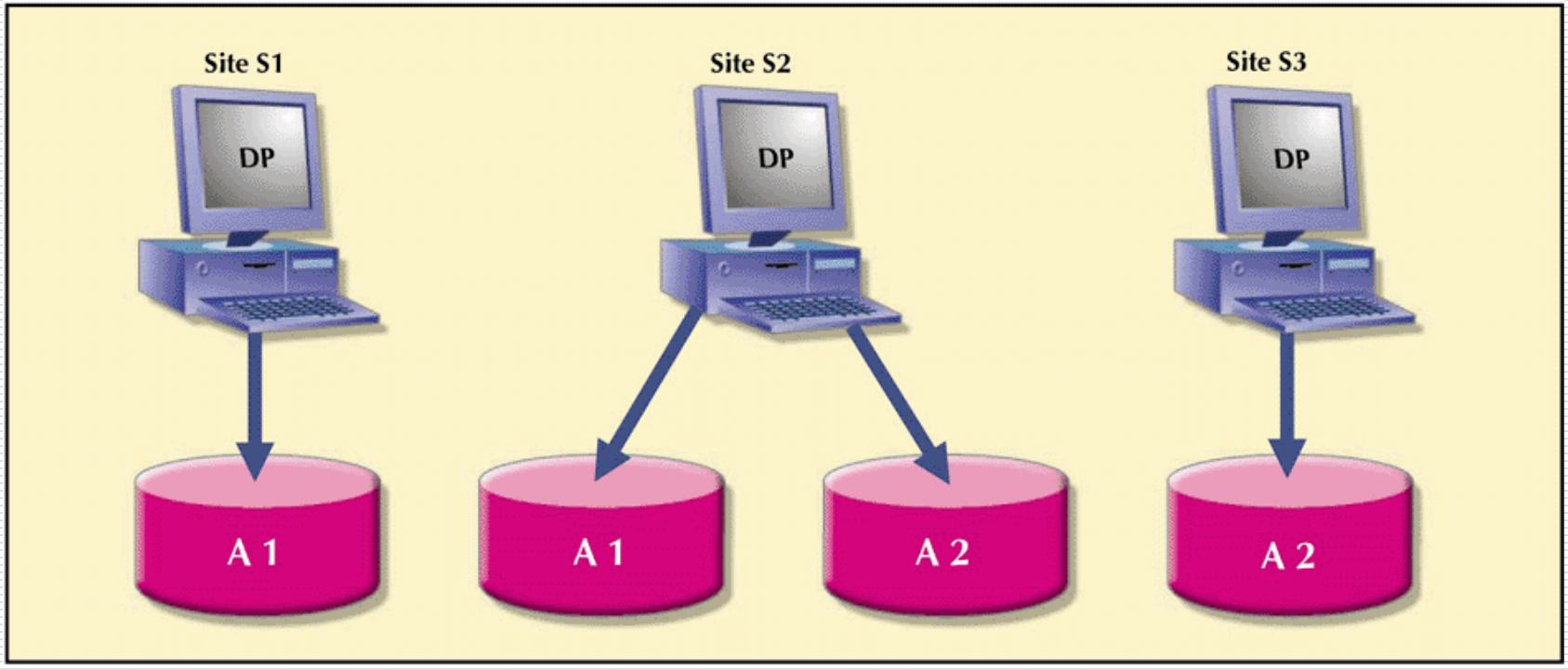
CUS_NUM	CUS_LIMIT	CUS_BAL	CUS_RATING	CUS_DUE
11	$6,000.00	$1,200.00	1	$0.00
13	$6,000.00	$5,890.00	3	$1,090.00
14	$1,200.00	$550.00	1	$0.00

Data Replication

- Storage of data copies at multiple sites served by a computer network
- Fragment copies can be stored at several sites to serve specific information requirements
  - Can enhance data availability and response time
  - Can help to reduce communication and total query costs
  - Imposes additional processing overhead
    - Which copy do you read when submitting a query
    - All copies must be updated when a write occurs

Data Replication

FIGURE 10.20 DATA REPLICATION



Replication Scenarios

- Fully replicated database:
  - Stores multiple copies of *each* database fragment at multiple sites
  - Can be impractical due to amount of overhead
- Partially replicated database:
  - Stores multiple copies of *some* database fragments at multiple sites
  - Most DDBMSs are able to handle the partially replicated database well
- Unreplicated database:
  - Stores each database fragment at a single site
  - No duplicate database fragments
  - Database size, usage frequency and costs (performance, overhead, management) influence the decision to replicate

Data Allocation

- Deciding where to locate data
- Allocation strategies:
  - Centralized data allocation
    - Entire database is stored at one site
  - Partitioned data allocation
    - Database is divided into several disjointed parts (fragments) and stored at several sites
  - Replicated data allocation
    - Copies of one or more database fragments are stored at several sites
- Data distribution over a computer network is achieved through data partition, data replication, or a combination of both

Client/Server vs. DDBMS

- Way in which computers interact to form a system
- Features a *user* of resources, or a client, and a *provider* of resources, or a server
- Can be used to implement a DBMS in which the client is the TP and the server is the DP
  - The client interacts with the end user and sends a request to the server.
  - The server receives, schedules and executes the request, selecting only those records that are needed by the client.
  - The server sends the data to the client only when the client requests the data.

Client/Server Advantages

- Less expensive than alternate minicomputer or mainframe solutions
- Allow end user to use microcomputer's GUI, thereby improving functionality and simplicity
- More people with PC skills than with mainframe skills in the job market
- PC is well established in the workplace
- Numerous data analysis and query tools exist to facilitate interaction with DBMSs available in the PC market
- Considerable cost advantage to offloading applications development from the mainframe to powerful PCs

Client/Server Disadvantages

- Creates a more complex environment, in which different platforms (LANs, operating systems, and so on) are often difficult to manage
- An increase in the number of users and processing sites often paves the way for security problems
- Possible to spread data access to a much wider circle of users→ increases demand for people with broad knowledge of computers and software→ increases burden of training and cost of maintaining the environment

C. J. Date's Twelve Commandments for Distributed Databases

1. Local site independence
2. Central site independence
3. Failure independence
4. Location transparency
5. Fragmentation transparency
6. Replication transparency
7. Distributed query processing
8. Distributed transaction processing
9. Hardware independence
10. Operating system independence
11. Network independence
12. Database independence